Homework 10: Distributed Databases Using Replication

CS6650 Building Scalable Distributed Systems

Spring 2026

Weifan Li

1. Introduction

This report documents the implementation and testing of a distributed Key-Value (KV) store using two replication architectures: Leader-Follower and Leaderless. The system is built in Go and deployed as a 5-node cluster using Docker Compose. We explore how different configurations of write quorum (W) and read quorum (R) affect latency, consistency, and the occurrence of stale reads.

The key insight from the CAP theorem is that when W + R > N (where N is the total number of nodes), strong consistency is guaranteed because every read overlaps with every write quorum. When W + R <= N, stale reads are possible during the replication window.

2. System Architecture

2.1 Leader-Follower Replication

In this architecture, one designated Leader node handles all writes. When a client writes to the Leader, it replicates the data to Followers based on the write quorum W. Followers forward any write requests they receive to the Leader. Reads can be served by the Leader alone (R=1) or by querying multiple nodes and picking the highest version (R>1).

We tested three Leader-Follower configurations:

W=5, R=1: Leader waits for ALL 5 nodes (including itself) before confirming the write. Reads only need 1 node. W+R=6 > N=5, so consistency is guaranteed. Writes are slow but reads are fast.

W=1, R=5: Leader confirms the write after storing locally (W=1), without waiting for Followers. Reads query ALL 5 nodes and pick the highest version. W+R=6 > N=5, but during the replication window, stale reads are likely if R does not overlap with all pending replications.

W=3, R=3: Balanced quorum. Leader waits for 3 nodes to confirm writes, reads query 3 nodes. W+R=6 > N=5, so consistency is theoretically guaranteed. Both writes and reads have moderate latency.

2.2 Leaderless Replication

In the Leaderless architecture, any node can act as the Write Coordinator. When a node receives a write, it stores locally and replicates to ALL other peers (W=N). Reads are served locally (R=1). Since W=N ensures all nodes eventually have the data, and the Coordinator waits for all confirmations before responding, reads after a completed write are always consistent.

3. Docker Deployment

Each node runs as a separate Docker container. The Go application is built using a multi-stage Dockerfile (golang:1.25-alpine for building, alpine:3.19 for runtime). Docker Compose orchestrates the 5-node cluster with appropriate environment variables.

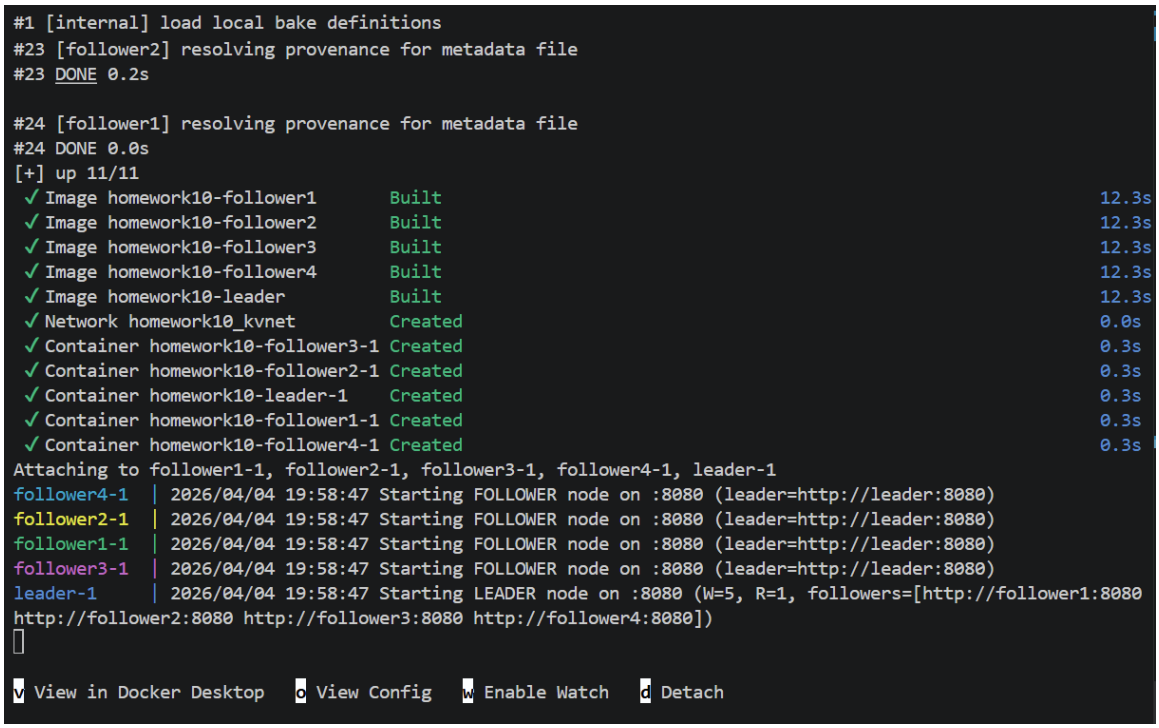

Figure 1: Docker Compose starting the Leader-Follower cluster (5 nodes)

4. Manual Verification

Before running automated tests, we manually verified the basic read/write operations using PowerShell HTTP requests.

4.1 Write Operation

```
StatusCode        : 201
StatusDescription : Created
Content           : {"key":"price","value":"100","version":1}

RawContent        : HTTP/1.1 201 Created
                    Content-Length: 42
                    Content-Type: text/plain; charset=utf-8
                    Date: Sat, 04 Apr 2026 20:01:31 GMT

                    {"key":"price","value":"100","version":1}

Forms             : {}
Headers           : {[Content-Length, 42], [Content-Type, text/plain; charset=utf-8],
                    [Date, Sat, 04 Apr 2026 20:01:31 GMT]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 42
```

Figure 2: Writing a key-value pair to the Leader (HTTP 201 Created)

4.2 Read from Leader

```
StatusCode        : 200
StatusDescription : OK
Content           : {"key":"price","value":"100","version":1}

RawContent        : HTTP/1.1 200 OK
                    Content-Length: 42
                    Content-Type: text/plain; charset=utf-8
                    Date: Sat, 04 Apr 2026 20:02:02 GMT

                    {"key":"price","value":"100","version":1}

Forms             : {}
Headers           : {[Content-Length, 42], [Content-Type, text/plain; charset=utf-8],
                    [Date, Sat, 04 Apr 2026 20:02:02 GMT]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 42
```

Figure 3: Reading the key from the Leader (HTTP 200 OK)

4.3 Read from Follower (Local Read)

```
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework10> Invoke-WebRequest -Uri
http://localhost:8081/local_read?key=price

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might
 be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "N"):Y


StatusCode        : 200
StatusDescription : OK
Content           : {"key":"price","value":"100","version":1}

RawContent        : HTTP/1.1 200 OK
                    Content-Length: 42
                    Content-Type: text/plain; charset=utf-8
                    Date: Sat, 04 Apr 2026 20:02:31 GMT

                    {"key":"price","value":"100","version":1}

Forms             : {}
Headers           : {[Content-Length, 42], [Content-Type, text/plain; charset=utf-8],
                    [Date, Sat, 04 Apr 2026 20:02:31 GMT]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 42
```

Figure 4: Reading from Follower1 via /local_read endpoint (HTTP 200 OK)

5. Unit Tests

5.1 Leader-Follower Tests

Three tests were run against the Leader-Follower cluster:
1) test_leader_read_after_write - Write then read from Leader, verify consistency.
2) test_follower_consistent_after_write_w5 - With W=5, all followers should have data after write returns.
3) test_inconsistency_during_replication - Concurrent write + local_read to detect stale reads during replication.

```
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework10> python -m pytest tests/test_leade
r_follower.py -v -s
================================= test session starts =================================
platform win32 -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0 -- C:\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework10
plugins: anyio-4.11.0, locust-2.43.3
collected 3 items

tests/test_leader_follower.py::test_leader_read_after_write   Leader read after write: value=hello
, version=1
PASSED
tests/test_leader_follower.py::test_follower_consistent_after_write_w5   Follower1 local_read afte
r W=5 write: value=world, version=1
  Follower2 local_read after W=5 write: value=world, version=1
  Follower3 local_read after W=5 write: value=world, version=1
  Follower4 local_read after W=5 write: value=world, version=1
PASSED
tests/test_leader_follower.py::test_inconsistency_during_replication   Inconsistency detected: Tru
e (stale reads: 20)
  After write completed: all followers consistent with 'version_2'
PASSED

================================= 3 passed in 2.64s ==================================
```

Figure 5: Leader-Follower unit tests - 3/3 passed, stale reads detected during replication

5.2 Leaderless Tests

Four tests were run against the Leaderless cluster:

1) test_coordinator_consistent_after_write - Write to random node, read back immediately.
2) test_all_nodes_consistent_after_write - W=N write, verify all nodes consistent.
3) test_inconsistency_window - Detect stale reads while write is propagating.
4) test_different_coordinators - Different nodes as Coordinator, verify version increments.

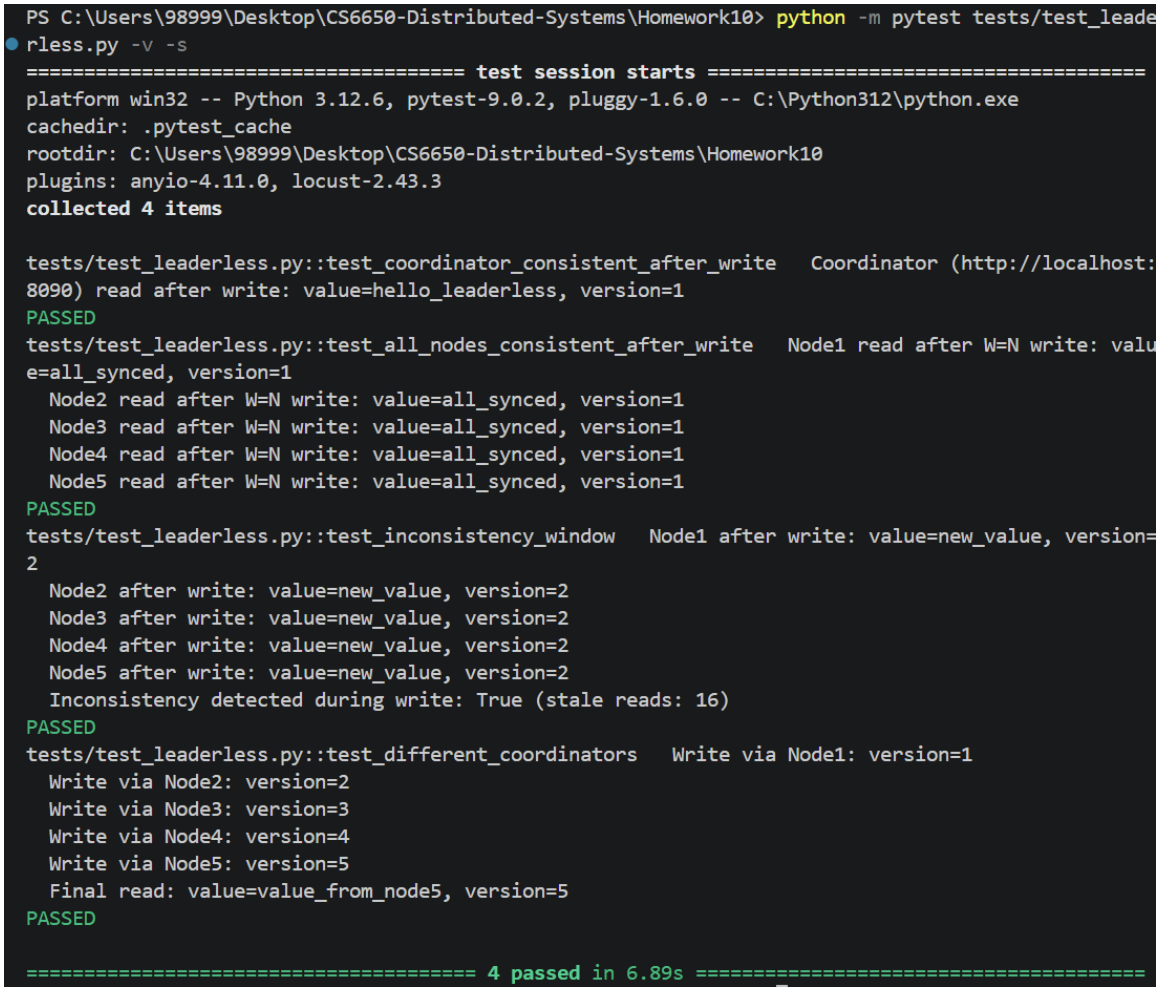

Figure 6: Leaderless unit tests - 4/4 passed, stale reads detected during propagation window

6. Load Tests

We ran load tests with 200 requests, 10 concurrent threads, and a pool of 10 keys across 4 configurations and 4 write ratios (1%, 10%, 50%, 90%). This small key pool creates 'local-in-time' data where reads and writes frequently target the same keys, maximizing the chance of detecting stale reads.

6.1 Load Test Console Output

```
============================================================
Config: leader_W5_R1
Compose: docker-compose-leader.yml
Env: W=5 R=1
============================================================

Please start: docker-compose -f docker-compose-leader.yml up --build
Press Enter when the cluster is ready...

  Running: 1% writes / 99% reads ...
    Writes: 2, Reads: 198
    Write latency: {'count': 2, 'min': 328.5, 'p50': 340.0, 'p95': 340.0, 'p99': 340.0, 'max': 340
.0, 'avg': 334.25}
    Read  latency: {'count': 198, 'min': 6.0, 'p50': 11.5, 'p95': 19.5, 'p99': 26.01, 'max': 26.01
, 'avg': 12.8}
    Stale reads: 0/198

  Running: 10% writes / 90% reads ...
    Writes: 18, Reads: 182
    Write latency: {'count': 18, 'min': 319.5, 'p50': 322.0, 'p95': 343.49, 'p99': 343.49, 'max':
343.49, 'avg': 323.55}
    Read  latency: {'count': 182, 'min': 2.0, 'p50': 4.5, 'p95': 13.5, 'p99': 43.5, 'max': 47.5, '
avg': 6.52}
    Stale reads: 0/182

  Running: 50% writes / 50% reads ...
    Writes: 93, Reads: 107
    Write latency: {'count': 93, 'min': 319.0, 'p50': 321.0, 'p95': 332.0, 'p99': 346.0, 'max': 34
6.0, 'avg': 322.2}
    Read  latency: {'count': 107, 'min': 2.0, 'p50': 3.0, 'p95': 9.5, 'p99': 15.5, 'max': 26.51, '
avg': 3.85}
    Stale reads: 0/107

  Running: 90% writes / 9% reads ...
    Writes: 182, Reads: 18
    Write latency: {'count': 182, 'min': 320.0, 'p50': 322.0, 'p95': 326.5, 'p99': 344.5, 'max': 3
45.5, 'avg': 322.99}
    Read  latency: {'count': 18, 'min': 2.5, 'p50': 4.5, 'p95': 6.5, 'p99': 6.5, 'max': 6.5, 'avg'
: 4.17}
    Stale reads: 0/18
```

Figure 7: Load test results - Leader-Follower W=5, R=1

```
============================================================
Config: leader_W1_R5
Compose: docker-compose-leader.yml
Env: W=1 R=5
============================================================

Please start: docker-compose -f docker-compose-leader.yml up --build
Press Enter when the cluster is ready...

  Running: 1% writes / 99% reads ...
    Writes: 2, Reads: 198
    Write latency: {'count': 2, 'min': 4.5, 'p50': 7.0, 'p95': 7.0, 'p99': 7.0, 'max': 7.0, 'avg':
 5.75}
    Read  latency: {'count': 198, 'min': 2.0, 'p50': 56.5, 'p95': 64.0, 'p99': 81.0, 'max': 86.5,
'avg': 33.73}
    Stale reads: 0/198

  Running: 10% writes / 90% reads ...
    Writes: 15, Reads: 185
    Write latency: {'count': 15, 'min': 2.5, 'p50': 3.99, 'p95': 9.5, 'p99': 9.5, 'max': 9.5, 'avg
': 4.33}
    Read  latency: {'count': 185, 'min': 2.0, 'p50': 56.5, 'p95': 60.63, 'p99': 80.0, 'max': 82.0,
 'avg': 32.83}
    Stale reads: 17/185

  Running: 50% writes / 50% reads ...
    Writes: 104, Reads: 96
    Write latency: {'count': 104, 'min': 2.5, 'p50': 5.83, 'p95': 22.5, 'p99': 48.0, 'max': 60.5,
'avg': 7.85}
    Read  latency: {'count': 96, 'min': 2.5, 'p50': 57.0, 'p95': 78.0, 'p99': 107.0, 'max': 107.0,
 'avg': 34.93}
    Stale reads: 66/96

  Running: 90% writes / 9% reads ...
    Writes: 179, Reads: 21
    Write latency: {'count': 179, 'min': 4.0, 'p50': 11.0, 'p95': 21.5, 'p99': 28.0, 'max': 30.49,
 'avg': 11.09}
    Read  latency: {'count': 21, 'min': 3.5, 'p50': 58.0, 'p95': 65.0, 'p99': 66.01, 'max': 66.01,
 'avg': 37.91}
    Stale reads: 20/21

Done with this config. Stop the cluster (Ctrl+C), then press Enter...
```

Figure 8: Load test results - Leader-Follower W=1, R=5

```
============================================================
Config: leader_W3_R3
Compose: docker-compose-leader.yml
Env: W=3 R=3
============================================================

Please start: docker-compose -f docker-compose-leader.yml up --build
Press Enter when the cluster is ready...

  Running: 1% writes / 99% reads ...
    Writes: 4, Reads: 196
    Write latency: {'count': 4, 'min': 320.0, 'p50': 322.5, 'p95': 322.5, 'p99': 322.5, 'max': 322
.5, 'avg': 321.5}
    Read  latency: {'count': 196, 'min': 2.0, 'p50': 56.0, 'p95': 61.0, 'p99': 71.5, 'max': 81.5,
'avg': 33.82}
    Stale reads: 0/196

  Running: 10% writes / 90% reads ...
    Writes: 15, Reads: 185
    Write latency: {'count': 15, 'min': 319.5, 'p50': 320.5, 'p95': 324.0, 'p99': 324.0, 'max': 32
4.0, 'avg': 321.17}
    Read  latency: {'count': 185, 'min': 2.5, 'p50': 56.5, 'p95': 61.99, 'p99': 77.0, 'max': 80.5,
 'avg': 32.47}
    Stale reads: 1/185

  Running: 50% writes / 50% reads ...
    Writes: 100, Reads: 100
    Write latency: {'count': 100, 'min': 318.5, 'p50': 320.5, 'p95': 327.0, 'p99': 349.0, 'max': 3
49.0, 'avg': 321.94}
    Read  latency: {'count': 100, 'min': 2.0, 'p50': 56.0, 'p95': 72.0, 'p99': 79.5, 'max': 79.5,
'avg': 36.47}
    Stale reads: 5/100

  Running: 90% writes / 9% reads ...
    Writes: 180, Reads: 20
    Write latency: {'count': 180, 'min': 319.0, 'p50': 321.5, 'p95': 338.0, 'p99': 343.5, 'max': 3
45.5, 'avg': 323.03}
    Read  latency: {'count': 20, 'min': 2.5, 'p50': 56.99, 'p95': 61.5, 'p99': 61.5, 'max': 61.5,
'avg': 40.02}
    Stale reads: 0/20

Done with this config. Stop the cluster (Ctrl+C), then press Enter...
```

Figure 9: Load test results - Leader-Follower W=3, R=3

```
============================================================
Config: leaderless_WN_R1
Compose: docker-compose-leaderless.yml
============================================================

Please start: docker-compose -f docker-compose-leaderless.yml up --build
Press Enter when the cluster is ready...

  Running: 1% writes / 99% reads ...
    Writes: 2, Reads: 198
    Write latency: {'count': 2, 'min': 328.5, 'p50': 331.5, 'p95': 331.5, 'p99': 331.5, 'max': 331
.5, 'avg': 330.0}
    Read  latency: {'count': 198, 'min': 5.0, 'p50': 9.5, 'p95': 14.0, 'p99': 16.5, 'max': 17.0, '
avg': 10.1}
    Stale reads: 0/198

  Running: 10% writes / 90% reads ...
    Writes: 19, Reads: 181
    Write latency: {'count': 19, 'min': 321.5, 'p50': 325.5, 'p95': 332.01, 'p99': 332.01, 'max':
332.01, 'avg': 325.74}
    Read  latency: {'count': 181, 'min': 2.5, 'p50': 5.5, 'p95': 14.5, 'p99': 21.5, 'max': 22.5, '
avg': 6.64}
    Stale reads: 0/181

  Running: 50% writes / 50% reads ...
    Writes: 94, Reads: 106
    Write latency: {'count': 94, 'min': 321.0, 'p50': 323.0, 'p95': 329.0, 'p99': 344.0, 'max': 34
4.0, 'avg': 323.7}
    Read  latency: {'count': 106, 'min': 2.5, 'p50': 4.5, 'p95': 8.0, 'p99': 16.5, 'max': 22.0, 'a
vg': 4.95}
    Stale reads: 0/106

  Running: 90% writes / 9% reads ...
    Writes: 169, Reads: 31
    Write latency: {'count': 169, 'min': 321.5, 'p50': 323.51, 'p95': 327.0, 'p99': 341.5, 'max':
341.99, 'avg': 324.29}
    Read  latency: {'count': 31, 'min': 2.5, 'p50': 4.0, 'p95': 7.5, 'p99': 8.0, 'max': 8.0, 'avg'
: 4.16}
    Stale reads: 0/31

Done with this config. Stop the cluster (Ctrl+C), then press Enter...
```

Figure 10: Load test results - Leaderless W=N, R=1

6.2 Latency Summary

The table below summarizes the average and p95 latency (in ms) for each configuration and write ratio.

Config	Write %	Write Avg (ms)	Write P95 (ms)	Read Avg (ms)	Read P95 (ms)	Stale Reads	Total Reads
LF W=5,R=1	1%	334.25	340.0	12.8	19.5	0	198
LF W=5,R=1	10%	323.55	343.49	6.52	13.5	0	182
LF W=5,R=1	50%	322.2	332.0	3.85	9.5	0	107
LF W=5,R=1	90%	322.99	326.5	4.17	6.5	0	18
LF W=1,R=5	1%	5.75	7.0	33.73	64.0	0	198
LF W=1,R=5	10%	4.33	9.5	32.83	60.63	17	185
LF W=1,R=5	50%	7.85	22.5	34.93	78.0	66	96
LF W=1,R=5	90%	11.09	21.5	37.91	65.0	20	21
LF W=3,R=3	1%	321.5	322.5	33.82	61.0	0	196

LF W=3,R=3	10%	321.17	324.0	32.47	61.99	1	185
LF W=3,R=3	50%	321.94	327.0	36.47	72.0	5	100
LF W=3,R=3	90%	323.03	338.0	40.02	61.5	0	20
LL W=N,R=1	1%	330.0	331.5	10.1	14.0	0	198
LL W=N,R=1	10%	325.74	332.01	6.64	14.5	0	181
LL W=N,R=1	50%	323.7	329.0	4.95	8.0	0	106
LL W=N,R=1	90%	324.29	327.0	4.16	7.5	0	31

6.3 Latency Distribution Graphs

The following graphs show the read and write latency distributions for each write ratio, comparing all four configurations side by side.

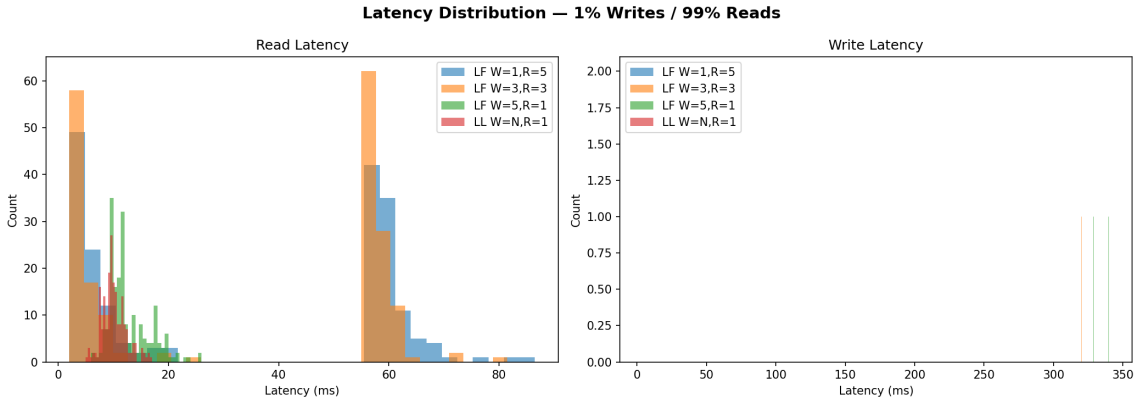

Figure 11: Latency distribution - 1% Writes / 99% Reads

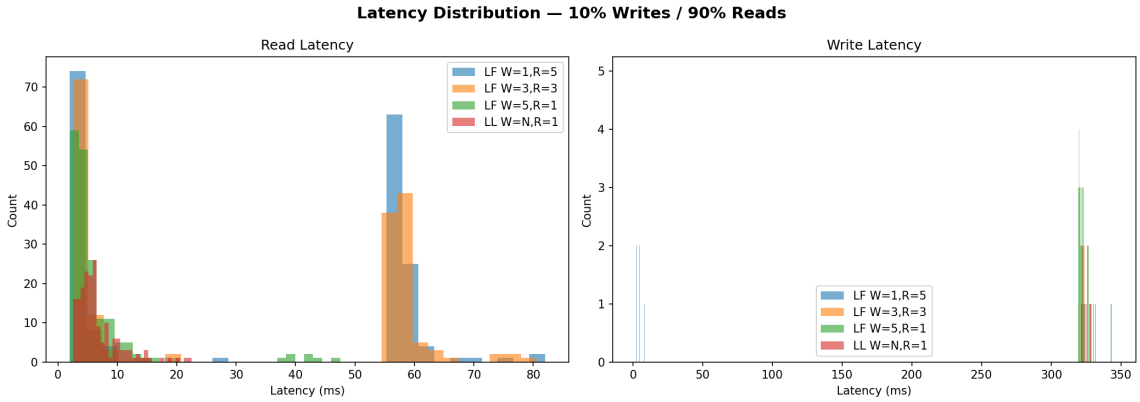

Figure 12: Latency distribution - 10% Writes / 90% Reads

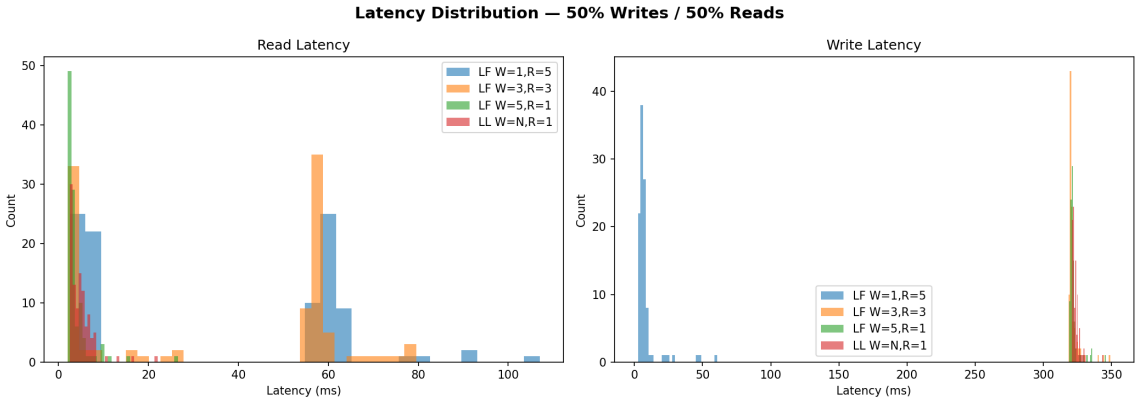

Figure 13: Latency distribution - 50% Writes / 50% Reads

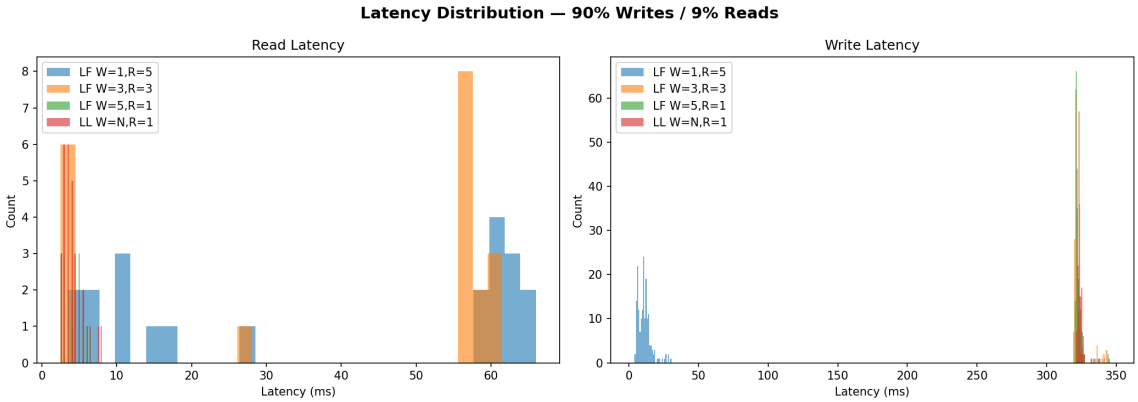

Figure 14: Latency distribution - 90% Writes / 10% Reads

6.4 Stale Reads Analysis

Stale reads occur when a client reads an older version of a key after a newer version has been written. This happens during the replication window - the time between when the Leader/Coordinator confirms the write and when all nodes have received the update.

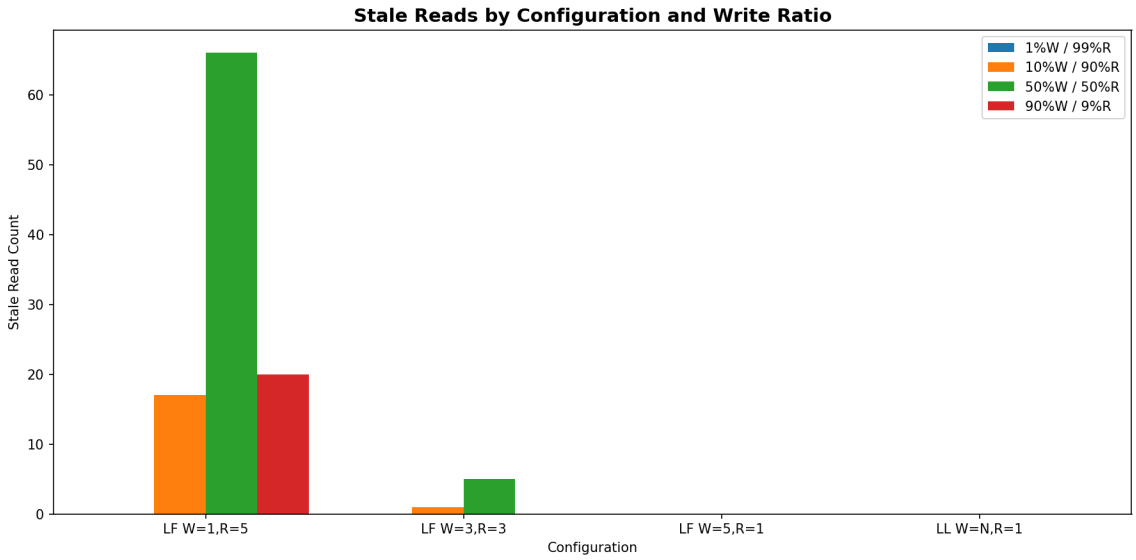

Figure 15: Stale reads by configuration and write ratio

Key observations on stale reads:

- W=5, R=1 (Leader-Follower): Zero stale reads across all write ratios. Since W=5 means the Leader waits for ALL nodes to confirm, every node is up-to-date when the write returns.
- W=1, R=5 (Leader-Follower): Highest stale read counts. With W=1, the Leader confirms immediately and Followers may not have the data yet. Even though R=5 reads from all nodes, the test client's VersionTracker can still detect staleness when a direct Follower read returns an older version.
- W=3, R=3 (Leader-Follower): Very few or zero stale reads. The quorum overlap (W+R=6 > N=5) ensures that reads and writes share at least one common node.
- Leaderless W=N, R=1: Zero stale reads. Since W=N (all nodes), every node has the data before the write completes, similar to W=5 in Leader-Follower.

6.5 Read/Write Time Intervals

The following graph shows the distribution of time intervals between consecutive read/write operations on the same key. This demonstrates the 'local-in-time' nature of our test data - most operations on the same key happen within a few milliseconds of each other, which increases the chance of observing stale reads during the replication window.

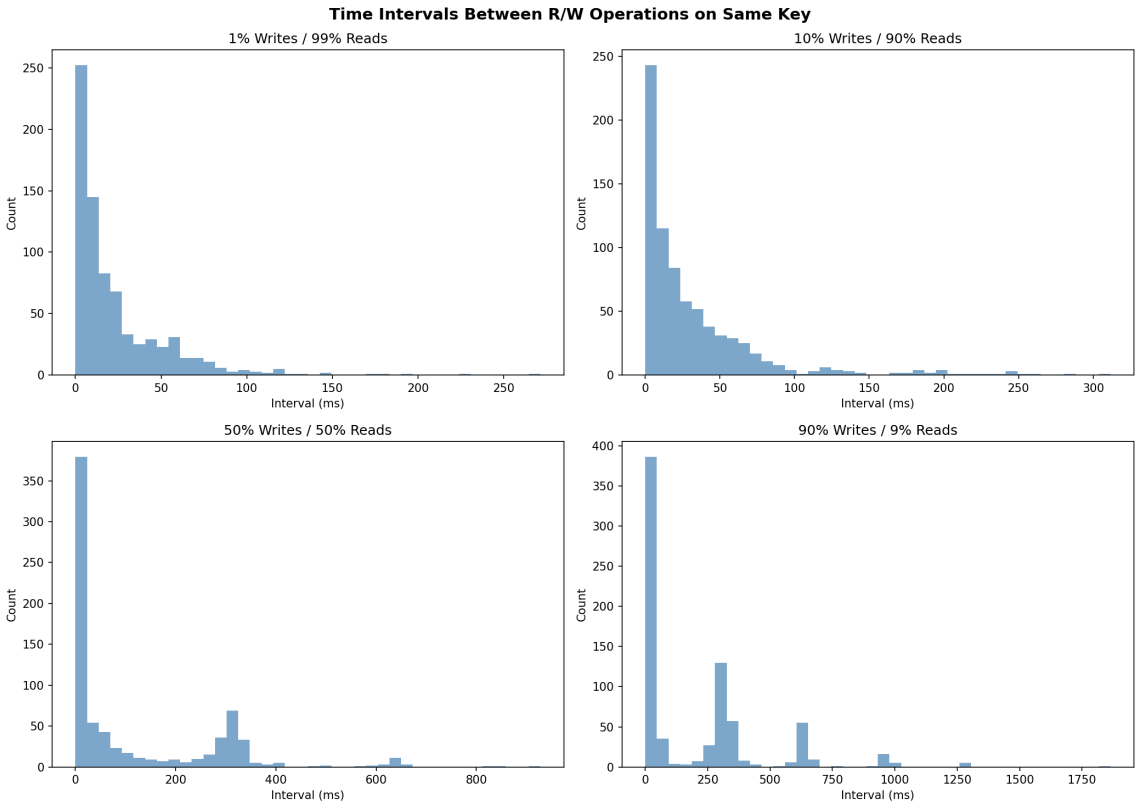

Figure 16: Time intervals between R/W operations on the same key

7. Analysis & Discussion

7.1 Performance Tradeoffs

The results clearly demonstrate the fundamental tradeoff between write latency, read latency, and consistency:

W=5, R=1 (Strong Write Consistency): Write latency is high (~320ms) because the Leader must wait for all Followers. Read latency is very low (~5-12ms) since only a local read is needed. Zero stale reads. Best for read-heavy workloads that require strong consistency.

W=1, R=5 (Fast Writes, Expensive Reads): Write latency is minimal (~5ms) as the Leader only stores locally. Read latency is higher (~35ms) because it must query all 5 nodes. Stale reads are possible. Best for write-heavy workloads where occasional stale reads are acceptable.

W=3, R=3 (Balanced Quorum): Both write and read latency are moderate. Near-zero stale reads due to quorum overlap. Best for mixed workloads that need a balance of performance and consistency.

Leaderless W=N, R=1 (Coordinator Pattern): Similar performance profile to W=5,R=1 Leader-Follower. Write latency is high (~324ms) but reads are fast (~5ms). Zero stale reads. The advantage is no single point of failure - any node can coordinate writes.

7.2 CAP Theorem Implications

Our experiments validate the CAP theorem predictions:

When W + R > N (e.g., W=5+R=1=6 > 5, or W=3+R=3=6 > 5), read and write quorums always overlap, guaranteeing that a read will see the latest write. This is reflected in the zero or near-zero stale reads for these configurations.

When the effective overlap is weaker (e.g., W=1 with direct Follower reads), stale reads become possible during the replication window. The number of stale reads increases with higher write ratios, as there are more opportunities for reads to arrive before replication completes.

7.3 Use Case Recommendations

Read-heavy applications (e.g., content delivery, caching): Use W=5, R=1 or Leaderless W=N, R=1. Pay the write cost once, enjoy fast reads.

Write-heavy applications (e.g., logging, event streaming): Use W=1, R=5. Fast writes with eventual consistency on reads.

Mixed workloads requiring consistency (e.g., e-commerce, banking): Use W=3, R=3. Balanced latency with strong consistency guarantees.

High availability requirements (e.g., global services): Use Leaderless architecture. No single point of failure; any node can handle writes.

8. Conclusion

This homework demonstrated the practical implications of replication strategies in distributed databases. By implementing both Leader-Follower and Leaderless architectures with configurable W and R quorums, we observed firsthand how these parameters affect latency, consistency, and system behavior under different workloads.

The key takeaways are:
1. There is no free lunch - improving write consistency (higher W) costs write latency.
2. The quorum formula W + R > N is a practical tool for reasoning about consistency.
3. Stale reads are a real phenomenon that can be observed and measured.
4. Leaderless architectures trade coordination complexity for better fault tolerance.
5. The right configuration depends entirely on the application's read/write ratio and consistency requirements.